

SDA Project 1: AutoComplete

Pavlov Aleksandr s2400691
Kostenko Oleksandr s2508297

Table des matières

1	Analyse empirique de tri	3
2	Analyse et choix de l'algorithme	3
2.1	Résultats et complexité	3
2.2	Algorithme approprié	3
3	Analyse de complexité	4
3.0.1	Notation	4
3.1	Approche 1 : Recherche linéaire	4
3.1.1	Complexité en temps	4
3.1.2	Complexité totale	4
3.1.3	Complexité en espace	4
3.2	Approche 2 : Tri lexicographique + recherche binaire + tri des résultats	4
3.2.1	Complexité en temps	5
3.2.2	Complexité totale	5
3.2.3	Complexité en espace	5
3.2.4	Complexité totale	6
3.3	Approche 3 : Tri lexicographique + recherche binaire + tas binaire	6
3.3.1	Complexité en temps	6
3.3.2	Complexité totale	6
3.3.3	Complexité en espace	6
3.3.4	Complexité totale	7
4	Analyse empirique de l'auto-complétion	7
4.1	Dataset : actors_25k	7
4.2	Dataset : actors_250k	8
4.3	Dataset : actors_2500k	8
5	Discussion des résultats	8
6	Conclusion	9

Introduction

L'objectif est de trouver les k termes les plus pertinents parmi N à partir d'un préfixe.

Nous comparons trois méthodes : la recherche linéaire, le tri avec recherche binaire, et la file de priorité.

1 Analyse empirique de tri

Nous rapportons les temps de calcul pour les algorithmes QuickSort (QS) et MergeSort (MS) sur les trois fichiers de données. Chaque fichier a été trié selon deux critères : l'ordre lexicographique l'ordre par poids.

TABLE 1 – Temps de calcul des algorithmes de tri (en secondes)			
Fichier	Critère	QuickSort (s)	MergeSort (s)
actors.25k.txt	Lexicographique	0.007394	0.007174
actors.25k.txt	Poids	0.001915	0.002683
actors.250k.txt	Lexicographique	0.113568	0.123857
actors.250k.txt	Poids	0.050852	0.031511
actors.2500k.txt	Lexicographique	2.071485	2.505856
actors.2500k.txt	Poids	1.412158	0.401059

2 Analyse et choix de l'algorithme

2.1 Résultats et complexité

Les tests confirment la complexité théorique en $O(N \log N)$. Quand on multiplie la taille des données par 10, le temps de calcul augmente de façon cohérente (environ 15 à 20 fois).

On note cependant que sur 2,5 millions de lignes, le MergeSort est 3,5 fois plus rapide que le QuickSort (0,40s contre 1,41s). Cela s'explique par la sensibilité du QuickSort au choix du pivot dans une liste déjà triée, alors que le MergeSort reste stable et constant peu importe l'ordre des données.

2.2 Algorithme approprié

Le tri fusion (MergeSort) est préféré.

Même si le QuickSort est un peu plus rapide, le MergeSort garantit une performance stable en $O(N \log N)$ dans tous les cas. Nos tests prouvent que le MergeSort est plus performant sur de gros volumes de données déjà triées.

Il faut noter que les performances du tri rapide dépendent du choix du pivot. Avec un autre pivot(nous avons utilisé l'élément le plus à droite), il peut être plus efficace.

3 Analyse de complexité

3.0.1 Notation

Soit :

- N : nombre total de termes,
- m : nombre de termes correspondant au préfixe de la requête,
- k : nombre de résultats à afficher.

3.1 Approche 1 : Recherche linéaire

Chaque requête parcourt l'ensemble des N termes pour sélectionner ceux correspondant au préfixe, puis conserve les k meilleurs selon leur poids.

3.1.1 Complexité en temps

Meilleur cas :

$$O(N)$$

Même dans le meilleur cas, tous les termes doivent être examinés.

Pire cas :

$$O(N)$$

Le filtrage est toujours linéaire.

3.1.2 Complexité totale

$$O(N)$$

3.1.3 Complexité en espace

$$O(1)$$

3.2 Approche 2 : Tri lexicographique + recherche binaire + tri des résultats

- Les termes sont triés lexicographiquement.
- Une recherche binaire permet d'identifier les m termes correspondant au préfixe.
- Les m termes sont ensuite triés par poids pour sélectionner les k meilleurs.

3.2.1 Complexité en temps

Tri initial :

- MergeSort : $O(N \log N)$
- QuickSort :
 - Meilleur cas : $O(N \log N)$
 - Pire cas : $O(N^2)$

Recherche binaire :

$$O(\log N)$$

Recherchons 2 éléments extrêmes

Extraction des m éléments :

$$O(m)$$

Tri des m éléments par poids :

- MergeSort : $O(m \log m)$
- QuickSort :
 - Meilleur cas : $O(m \log m)$
 - Pire cas : $O(m^2)$

3.2.2 Complexité totale

Avec MergeSort :

$$O(N \log N + m \log m)$$

Avec QuickSort :

- Meilleur cas :
$$O(N \log N + m \log m)$$
- Pire cas :
$$O(N^2 + m^2)$$

3.2.3 Complexité en espace

- MergeSort : $O(N)$
- QuickSort :
 - Meilleur cas : $O(\log N)$
 - Pire cas : $O(N)$

Stockage supplémentaire pour les résultats :

$$O(m)$$

- MergeSort : $O(m)$
- QuickSort :
 - Meilleur cas : $O(\log m)$
 - Pire cas : $O(m)$

3.2.4 Complexité totale

$$O(N)$$

3.3 Approche 3 : Tri lexicographique + recherche binaire + tas binaire

- Les termes sont triés lexicographiquement.
- Une recherche binaire identifie les m termes du préfixe.
- Une structure de tas binaire (heap) est utilisée pour sélectionner les k meilleurs selon leur poids.

3.3.1 Complexité en temps

Tri initial :

- MergeSort : $O(N \log N)$
- QuickSort :
 - Meilleur cas : $O(N \log N)$
 - Pire cas : $O(N^2)$

Recherche binaire :

$$O(\log N)$$

Recherchons 2 éléments extrêmes.

Parcours des m éléments avec tas de taille $\min(k, m)$:

$$O(m \log \min(k, m))$$

3.3.2 Complexité totale

Avec MergeSort :

$$O(N \log N + m \log \min(k, m))$$

Avec QuickSort :

- Meilleur cas :
$$O(N \log N + m \log \min(k, m))$$
- Pire cas :
$$O(N^2 + m \log \min(k, m))$$

3.3.3 Complexité en espace

- MergeSort : $O(N)$
- QuickSort :
 - Meilleur cas : $O(\log N)$
 - Pire cas : $O(N)$

Espace supplémentaire pour le tas :

$$O(\min(k, m))$$

3.3.4 Complexité totale

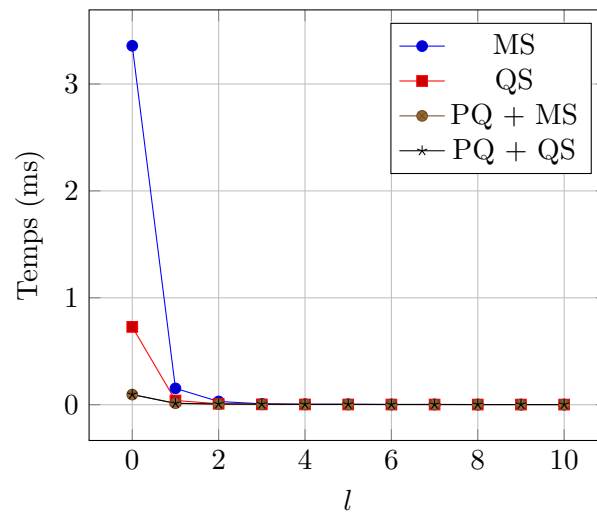
$$O(N)$$

4 Analyse empirique de l'auto-complétion

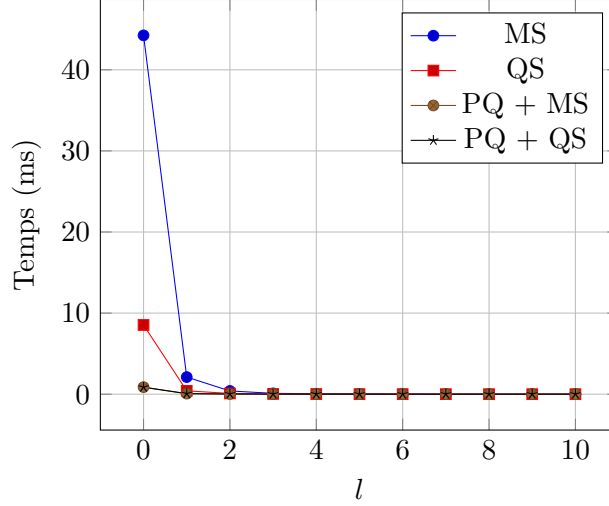
Nous avons fixé $k = 20$ et mesuré le temps moyen de `acComplete` sur 100 essais pour des longueurs de requête l allant de 0 à 10.

- MergeSort
- QuickSort
- PQ + MergeSort
- PQ + QuickSort

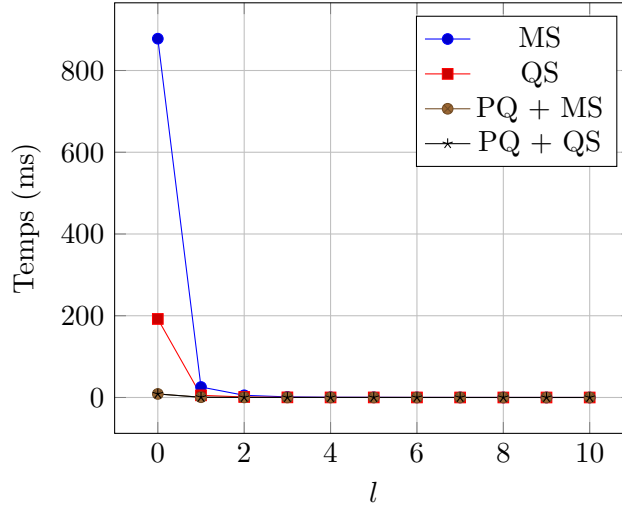
4.1 Dataset : actors_25k



4.2 Dataset : actors_250k



4.3 Dataset : actors_2500k



5 Discussion des résultats

Les graphiques montrent qu'à partir de $l \geq 2$, les différences entre les approches deviennent faibles.

La longueur du préfixe l influence directement le temps d'exécution. Quand le préfixe est court, beaucoup de termes correspondent à la requête, ce qui augmente le nombre d'éléments à traiter et donc le coût du tri selon le poids. À l'inverse, lorsque le préfixe est plus long, la recherche binaire permet d'éliminer plus de termes dès le début, ce qui réduit le nombre de candidats m et accélère le traitement.

Ainsi, quand l augmente, les performances s'améliorent car il y a moins de données à parcourir ou à trier.

Les graphiques confirment que le paramètre m est le facteur principal de la complexité temporelle. On observe que le temps d'exécution diminue nettement quand l augmente, ce qui réduit m .

6 Conclusion

Parmi les approches étudiées, l'utilisation d'une file de priorité (PQ) combinée à une recherche par préfixe est la plus efficace pour traiter les requêtes.

Les méthodes basées sur le tri sont moins performantes, car elles nécessitent de trier l'ensemble des mots correspondants, ce qui entraîne un coût plus élevé.

L'approche avec une file de priorité permet de sélectionner les meilleurs éléments sans effectuer un tri complet. Elle traite les résultats en $O(m \log \min(k, m))$, ce qui évite de trier les m éléments. Cette méthode est particulièrement adaptée au contexte où le nombre de résultats à afficher k est généralement bien inférieur au nombre de termes correspondants m .